In [29]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import datetime
```

In [ ]:
```
df = pd.read_csv("C:\\Users\\pande\\OneDrive\\Desktop\\Task 2\\data-export (1).csv")
```

In [10]:
```
df.head()
```

Out[10]:

	# ---------- ---------- ---------- ---	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unname
0	Session primary channel group (Default channel...	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessi per u
1	Direct	2024041623	237	300	144	47.526666666666700	0.6075949367088
2	Organic Social	2024041719	208	267	132	32.09737827715360	0.6346153846153
3	Direct	2024041723	188	233	115	39.93991416309010	0.6117021276595
4	Organic Social	2024041718	187	256	125	32.16015625	0.6684491978609

In [11]:
```
df.columns = df.iloc[0]
```

In [12]:
```
df.head()
```

Out[12]:

	Session primary channel group (Default channel group)	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	I
0	Session primary channel group (Default channel...	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	
1	Direct	2024041623	237	300	144	47.526666666666700	0.6075949367088610	4
2	Organic Social	2024041719	208	267	132	32.09737827715360	0.6346153846153850	4

	Session primary channel group (Default channel group)	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	I
3	Direct	2024041723	188	233	115	39.93991416309010	0.6117021276595740	4
4	Organic Social	2024041718	187	256	125	32.16015625	0.6684491978609630	

In [13]:
```
df = df.drop(index = 0).reset_index(drop = True)
```

In [14]:
```
df.head()
```

Out[14]:

	Session primary channel group (Default channel group)	Date + hour (YYYYMMDDHH)	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	E
0	Direct	2024041623	237	300	144	47.526666666666700	0.6075949367088610	4.
1	Organic Social	2024041719	208	267	132	32.09737827715360	0.6346153846153850	4.
2	Direct	2024041723	188	233	115	39.93991416309010	0.6117021276595740	4.
3	Organic Social	2024041718	187	256	125	32.16015625	0.6684491978609630	
4	Organic Social	2024041720	175	221	112	46.918552036199100	0.64	4.

In [15]:
```
df.columns=["Channel group","Datehours","Users","Sessions","Engaged sessions","Avera
```

In [16]:
```
df.head()
```

Out[16]:

	Channel group	Datehours	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events
0	Direct	2024041623	237	300	144	47.526666666666700	0.6075949367088610	4.67333
1	Organic Social	2024041719	208	267	132	32.09737827715360	0.6346153846153850	4.29588
2	Direct	2024041723	188	233	115	39.93991416309010	0.6117021276595740	4.58798
3	Organic Social	2024041718	187	256	125	32.16015625	0.6684491978609630	
4	Organic Social	2024041720	175	221	112	46.918552036199100	0.64	4.52941

In [22]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3182 entries, 0 to 3181
Data columns (total 10 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Channel group                         3182 non-null   object
 1   Datehours                             3182 non-null   datetime64[ns]
 2   Users                                 3182 non-null   object
 3   Sessions                              3182 non-null   object
 4   Engaged sessions                      3182 non-null   object
 5   Average engagement time per session   3182 non-null   object
 6   Engaged sessions per user             3182 non-null   object
 7   Events per session                    3182 non-null   object
 8   Engagement rate                       3182 non-null   object
 9   Event count                           3182 non-null   object
dtypes: datetime64[ns](1), object(9)
memory usage: 248.7+ KB
```

In [21]:
```
df["Datehours"] = pd.to_datetime(df["Datehours"],format ="%Y%m%d%H",errors ="coerce"
```

In [23]:
```
df.head()
```

Out[23]:

	Channel group	Datehours	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events p
0	Direct	2024-04-16 23:00:00	237	300	144	47.526666666666700	0.6075949367088610	4.673333
1	Organic Social	2024-04-17 19:00:00	208	267	132	32.09737827715360	0.6346153846153850	4.295880
2	Direct	2024-04-17 23:00:00	188	233	115	39.93991416309010	0.6117021276595740	4.587982
3	Organic Social	2024-04-17 18:00:00	187	256	125	32.16015625	0.6684491978609630	
4	Organic Social	2024-04-17 20:00:00	175	221	112	46.918552036199100	0.64	4.529411

In [24]:
```
numeric_cols= df.columns.drop(["Channel group","Datehours"])
```

In [25]:
```
df[numeric_cols]=df[numeric_cols].apply(pd.to_numeric,errors ="coerce")
```

In [26]:
```
df["Hours"]=df["Datehours"].dt.hour
```

In [27]:
```
df.head()
```

Out[27]:

	Channel group	Datehours	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate	Ev co
0	Direct	2024-04-16 23:00:00	237	300	144	47.526667	0.607595	4.673333	0.480000	1
1	Organic Social	2024-04-17 19:00:00	208	267	132	32.097378	0.634615	4.295880	0.494382	1
2	Direct	2024-04-17 23:00:00	188	233	115	39.939914	0.611702	4.587983	0.493562	1
3	Organic Social	2024-04-17 18:00:00	187	256	125	32.160156	0.668449	4.078125	0.488281	1
4	Organic Social	2024-04-17 20:00:00	175	221	112	46.918552	0.640000	4.529412	0.506787	1

In [28]:
```
df.describe()
```

Out[28]:

	Users	Sessions	Engaged sessions	Average engagement time per session	Engaged sessions per user	Events per session	Engagement rate
count	3182.000000	3182.000000	3182.000000	3182.000000	3182.000000	3182.000000	3182.000000
mean	41.935889	51.192646	28.325581	66.644581	0.606450	4.675969	0.503396
std	29.582258	36.919962	20.650569	127.200659	0.264023	2.795228	0.228206
min	0.000000	1.000000	0.000000	0.000000	0.000000	1.000000	0.000000
25%	20.000000	24.000000	13.000000	32.103034	0.561404	3.750000	0.442902
50%	42.000000	51.000000	27.000000	49.020202	0.666667	4.410256	0.545455
75%	60.000000	71.000000	41.000000	71.487069	0.750000	5.217690	0.633333
max	237.000000	300.000000	144.000000	4525.000000	2.000000	56.000000	1.000000

In [31]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3182 entries, 0 to 3181
Data columns (total 11 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Channel group                         3182 non-null   object
 1   Datehours                             3182 non-null   datetime64[ns]
 2   Users                                 3182 non-null   int64
 3   Sessions                              3182 non-null   int64
```

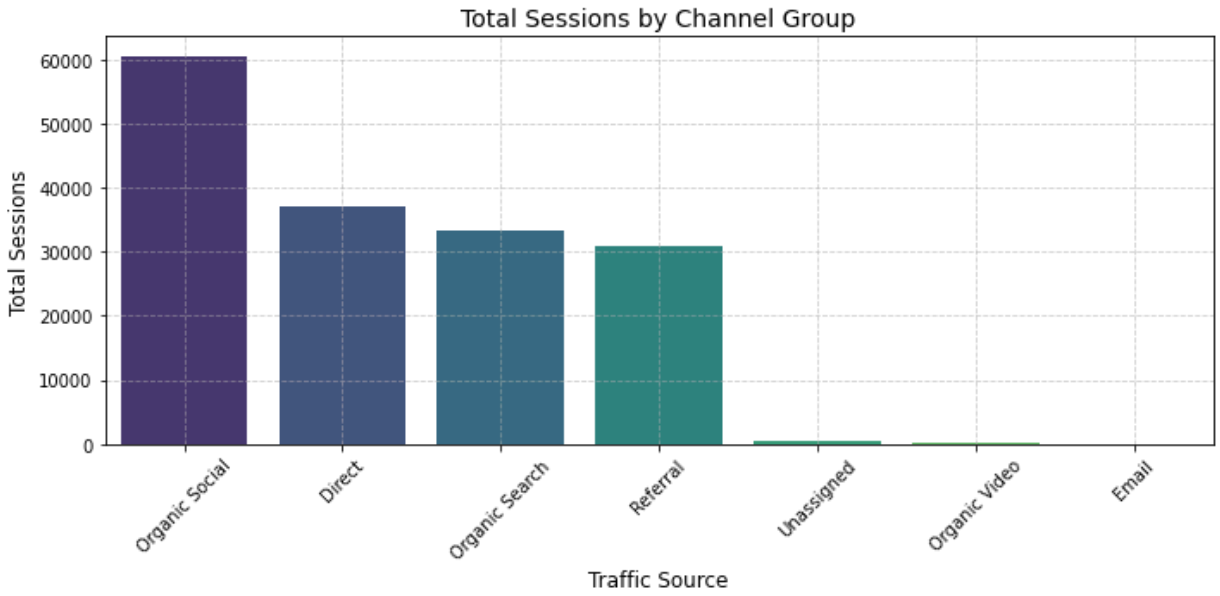
```
 4   Engaged sessions                       3182 non-null   int64
 5   Average engagement time per session    3182 non-null   float64
 6   Engaged sessions per user              3182 non-null   float64
 7   Events per session                     3182 non-null   float64
 8   Engagement rate                        3182 non-null   float64
 9   Event count                            3182 non-null   int64
 10  Hours                                  3182 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(5), object(1)
memory usage: 273.6+ KB
```

In [33]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Group and sum metrics by traffic channel
traffic_summary = df.groupby('Channel group')[['Users', 'Sessions', 'Event count']].

# Display the summary table
print("--- Top Traffic Sources Summary ---")
display(traffic_summary)

# Plotting the traffic sources
plt.figure(figsize=(10, 5))
sns.barplot(x=traffic_summary.index, y=traffic_summary['Sessions'], palette='viridis
plt.title('Total Sessions by Channel Group', fontsize=14)
plt.xlabel('Traffic Source', fontsize=12)
plt.ylabel('Total Sessions', fontsize=12)
plt.xticks(rotation=45)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

```
--- Top Traffic Sources Summary ---
```

Channel group	Users	Sessions	Event count
Organic Social	47572	60627	296631
Direct	30042	37203	156318
Organic Search	28387	33372	136957
Referral	26774	30990	177992
Unassigned	540	559	1932
Organic Video	123	141	1071
Email	2	3	10

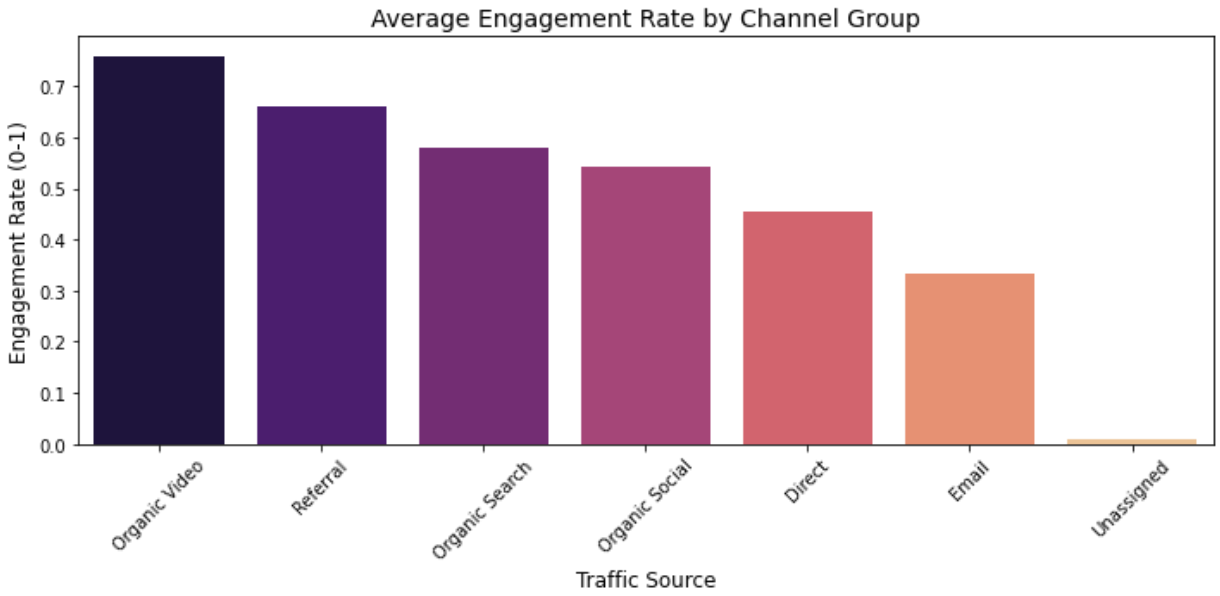


In [34]:

```
# Group by channel and get average engagement scores
engagement_summary = df.groupby('Channel group')[['Average engagement time per sessi

print("--- User Engagement by Channel ---")
display(engagement_summary)

# Plotting engagement rate vs channel group
plt.figure(figsize=(10, 5))
sns.barplot(x=engagement_summary.index, y=engagement_summary['Engagement rate'], pal
plt.title('Average Engagement Rate by Channel Group', fontsize=14)
plt.xlabel('Traffic Source', fontsize=12)
plt.ylabel('Engagement Rate (0-1)', fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

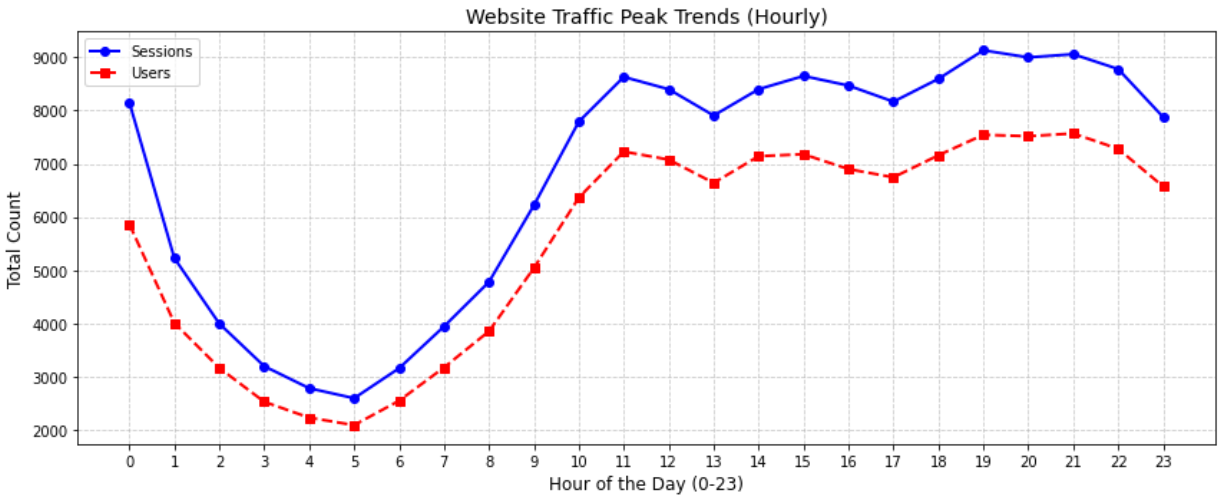
```
--- User Engagement by Channel ---
```

Channel group	Average engagement time per session	Engagement rate
Organic Video	180.360000	0.760000
Referral	92.660842	0.660882
Organic Search	47.005018	0.578906
Organic Social	53.493681	0.541180
Direct	45.533104	0.455723
Email	72.666667	0.333333
Unassigned	78.957923	0.007514



In [38]:

```
# Group by your extracted hour column
hourly_summary = df.groupby('Hours')[['Sessions', 'Users']].sum()

# Plotting hourly trends
plt.figure(figsize=(12, 5))
plt.plot(hourly_summary.index, hourly_summary['Sessions'], marker='o', color='b', li
plt.plot(hourly_summary.index, hourly_summary['Users'], marker='s', color='r', lines

plt.title('Website Traffic Peak Trends (Hourly)', fontsize=14)
plt.xlabel('Hour of the Day (0-23)', fontsize=12)
plt.ylabel('Total Count', fontsize=12)
plt.xticks(range(0, 24))
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
```



In [36]:

```
# Simple insight summary
top_channel = traffic_summary.index[0]
best_engaging_channel = engagement_summary.index[0]
peak_hour = hourly_summary['Sessions'].idxmax()

print("--- Business Insights Report ---")
print(f"1. Top Traffic Driver: '{top_channel}' brings in the highest volume of sessi
print(f"2. Best Audience Quality: '{best_engaging_channel}' has the highest overall
print(f"3. Operational Peak Time: Website traffic peaks at {peak_hour}:00 hrs. Sched
```

```
--- Business Insights Report ---
1. Top Traffic Driver: 'Organic Social' brings in the highest volume of sessions.
2. Best Audience Quality: 'Organic Video' has the highest overall engagement rate.
3. Operational Peak Time: Website traffic peaks at 19:00 hrs. Schedule critical updat
es or marketing campaigns outside this window.
```

In [ ]: